Auth & Bring-up Setup

Steps and commands only. Run from the repo root. `<like-this>` = placeholder.

---

0. Prereqs

- Docker + Docker Compose v2
- `jq`, `curl`, `openssl`, `python3`, `wg` / `wg-quick`
- WireGuard config for the JeMPI VPS (`10.8.0.1`, peer `202.51.71.81`)

---

1. Containers

```
docker compose up -d
docker compose ps
```

Running: `hapi-db`, `mongo-db`, `openhim-core`, `openhim-console`, `fhir`, `keycloak`, `mediator`, `admin-mediator`, `patient-portal-mediator`, `fhir-intake-mediator`.

> **Never `docker compose down`** (wipes Keycloak's H2). Use `stop` / `start`. Mediators crash-loop until steps 2 and 5 are done.

---

2. OpenHIM first login

2a. Accept the self-signed cert

1. Open https://localhost:8080/heartbeat, click through the warning.
2. CLI needs `-k`: `curl -k https://localhost:8080/heartbeat`

2b. Root password

1. Console http://localhost:9000, log in `root@openhim.org` / `openhim-password`.
2. Set a new password. Record it.

2c. Propagate to mediators

Set `OPENHIM_PASSWORD` in `mediator/.env`, `admin-mediator/.env`, `patient-portal-mediator/.env`, `fhir-intake-mediator/.env`, then:

```
docker compose up -d --force-recreate mediator admin-mediator patient-portal-mediator
        fhir-intake-mediator
```

> `restart` does **not** reload `.env` — use `up -d --force-recreate`.

Console → **Mediators**: all four green.

---

3. WireGuard tunnel to JeMPI

```
sudo wg-quick up wg0                                          # config at /etc/wireguard/
        wg0.conf
curl http://10.8.0.1:50000/JeMPI/countRecords                 # from host
docker exec mediator wget -qO- http://10.8.0.1:50000/JeMPI/countRecords   # from a
        container
```

If host works but container hangs:

```
sudo iptables -t nat -A POSTROUTING -s 172.16.0.0/12 -o wg0 -j MASQUERADE
sudo sysctl -w net.ipv4.ip_forward=1
```

Also ensure `wg0.conf` `AllowedIPs` includes `10.8.0.0/24`.

---

4. Keycloak realm

Admin Console http://localhost:8083, bootstrap admin `admin` / `admin`.

```
curl -fsS http://localhost:8083/realms/openhie/.well-known/openid-configuration |
        jq .issuer
```

If it prints `http://localhost:8083/realms/openhie`, the realm exists — go to §6. Otherwise:

- Realm dropdown → **Create realm** → name `openhie` → **Create**
- Or `./keycloak/setup.sh`

---

5. Realm key into OpenHIM

```
./keycloak/extract-realm-pubkey.sh
docker compose up -d --force-recreate openhim-core
docker compose restart mediator admin-mediator patient-portal-mediator fhir-intake-
        mediator
```

> A realm rebuild rotates Keycloak's RSA key → re-run this or every token fails `JWT could not be verified`.

OpenHIM env (already in `docker-compose.yml`): `authentication_jwt_algorithms` = `RS256` (plain string), `authentication_jwt_audience` = `account`, `authentication_jwt_issuer` = `http://localhost:8083/realms/openhie`.

Transaction-port errors:

Code	Meaning
401	token missing / bad signature / issuer or audience mismatch
403	Client mapped, but lacks a channel role
404	no channel matched the URL

---

6. Per-hospital client

One command per hospital — NHFR code + name. Creates the Organization, keypair, Keycloak client + `sub`-override mapper, and OpenHIM Client; verifies the token `sub`:

```
./keycloak/onboard-hospital.sh 3130400221 "Bharatpur Hospital"
```

Idempotent. Output gives the private-key path — hand it to the hospital, delete your copy.

System	Value
FHIR	`Organization.identifier` = `<nhfr-id>`; `Organization.id` = server UUID
Keycloak	client `<nhfr-id>`, `private_key_jwt`, token `sub` = `<nhfr-id>`
OpenHIM	Client **ID** = `<nhfr-id>`, role `np-fhir-clients`
Mediator	`X-OpenHIM-ClientID` = `<nhfr-id>` → `GET Organization?identifier=nhfr|<nhfr-id>`

Prereqs: §1, §2c, §5.

Just an Organization, no client: `./keycloak/create-org.sh <nhfr-id> "<name>"`.

> NHFR system string: deployed stack uses `http://mohp.gov.np/fhir/NamingSystem/nhfr` (NP Core's canonical is `https://nhfr.mohp.gov.np`).

> `sub` override rejected → `onboard-hospital.sh` fails at 4/5. Fallback: OpenHIM `clientID` stays the Keycloak UUID; add `FACILITY_REGISTRY_JSON={"<uuid>":"<nhfr-id>"}` to `mediator/.env` + `docker compose up -d --force-recreate mediator`.

---

7. Smoke test

```
TOKEN=$(./keycloak/get-jwt-client-token.sh 3130400221)

curl -k -sS -X POST https://localhost:5000/fhir/Patient \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/fhir+json' \
  -d '{"resourceType":"Patient","name":[{"family":"SHRESTHA","given":["KAMANA"]}],
       "gender":"female","birthDate":"2001-06-25",
       "telecom":[{"system":"phone","value":"9800000000"}]}'
```

`201` with a UUID `id` and `nepal-health-id` in `identifier` = the full chain works. Console → **Transactions** → the request → **orchestrations**.

Postman: `node keycloak/token-server.mjs`, then `http://localhost:4099/token/3130400221`.

---

8. FHIR Intake mediator (`/ingest`)

`POST /ingest` in front of the orchestrator:

Content-Type	Body	Action
`application/json`	one raw CDRS encounter row	

Content-Type	Body	Action
		validate → Patient + Encounter Bundle → forward
`application/fhir+json`	resource or `transaction` / `batch` Bundle	forward unmutated
other	—	`415`

Channel `NP FHIR Intake`, `^/ingest/?$`, `POST`, route `fhir-intake-mediator:4003`, role `OPENHIM_CHANNEL_ROLE` in `fhir-intake-mediator/.env` (`np-fhir-clients` to reuse §6 clients).

Raw-row behaviour: `403` if no `X-OpenHIM-ClientID`; `GET Organization?identifier=nhfr|<nhfr-id>` → `422` if 0 or >1 match; stamps `managingOrganization` / `serviceProvider` = the resolved `Organization/<uuid>`. Inbound `X-NHFR-ID` / `organization_id` ignored.

8a. Dump script

```
cd FHIR_mediator
cp .env.example .env          # KEYCLOAK_CLIENT_ID=<nhfr-id>, PRIVATE_KEY_FILE,
        SQLITE_PATH

DRY_RUN=1 python3 main.py      # preview
python3 main.py                # raw rows
python3 main.py --FHIR         # client-mapped Bundles
```

Progress bookmarked per `encounter_number`.

8b. Smoke test

```
TOKEN=$(./keycloak/get-jwt-client-token.sh 3130400221)

curl -k -sS -X POST https://localhost:5000/ingest \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"hospital_patient_number":"2405078","encounter_number":"SMOKE-1",
       "first_name":"CHANDRIKA","last_name":"KANDEL","sex":"Female",
       "department_type":"C","department_id":"22","department_name":"Family Medicine",
       "visit_date_ad_time":"2026-07-28 00:02:00.000"}'
```

`201`. A second POST of the same `encounter_number` must not create a duplicate.

---

9. Deploy on a VPS

Steps 1–8 assume everything is on `localhost`. On a VPS the token issuer must match across Keycloak, OpenHIM, and every client, or all tokens 401. Let `<vps>` = the VPS public host or domain.

9a. Keycloak reachable + stable issuer

`docker-compose.yml`, `keycloak` service:

- Port: `127.0.0.1:8083:8080` → `8083:8080`.
- Add env: `KC_HOSTNAME: https://<vps>:8083` (or the proxy URL from 9d).
- `start-dev` is fine for testing. For production also set `KC_DB: postgres` + DB env and drop `--import-realm`; `start-dev`'s H2 store is lost on any `docker compose down`.

9b. Point OpenHIM + clients at that issuer

- `docker-compose.yml`, `openhim-core`: `authentication_jwt_issuer: "https://<vps>:8083/realms/openhie"`.
- `FHIR_mediator/.env` (each hospital box): `KEYCLOAK_URL=https://<vps>:8083`.
- `iss` in a minted token = Keycloak's `KC_HOSTNAME`. All three must be byte-identical.

```
docker compose up -d --force-recreate keycloak openhim-core
./keycloak/extract-realm-pubkey.sh
docker compose up -d --force-recreate openhim-core
docker compose restart mediator admin-mediator patient-portal-mediator fhir-intake-
        mediator
```

9c. Firewall

Expose only:

Port	Service
5000	OpenHIM secure transaction port (client traffic)
8083	Keycloak
9000	OpenHIM Console — VPN / admin IP only

Keep internal (no host rule): `8080` OpenHIM API, `8081` HAPI, `5433` hapi-db, `27017` mongo, `50000` JeMPI.

9d. TLS

- Testing: `:5000` is already TLS with a self-signed cert. Clients keep `-k` / `OPENHIM_TRUST_SELF_SIGNED=true`.
- Production: put Caddy or nginx with a real cert in front of `:5000` and `:8083`; set `KC_HOSTNAME` / `authentication_jwt_issuer` to the proxy URL.

9e. Secrets

Change before exposing the host:

- Keycloak `KC_BOOTSTRAP_ADMIN_PASSWORD` (was `admin`).
- OpenHIM root password (§2b) → propagate to every `*/.env` (§2c).
- Production: `hapi-db` and `mongo-db` credentials too.

9f. JeMPI tunnel on the VPS

Run §3 on the VPS: `wg-quick up wg0`, `ip_forward`, the MASQUERADE rule. Without it the mediators cannot reach `10.8.0.1:50000`.

9g. Never `docker compose down`

Same rule as local — wipes Keycloak's H2 realm. Use `stop` / `start`. A VPS you depend on should be on `KC_DB=postgres` (9a) so this stops being fragile.

Quick reference

Task	Command
Create the realm	`./keycloak/setup.sh`
Create/update an Organization	`./keycloak/create-org.sh <nhfr-id> "<name>"`
Re-extract realm key	`./keycloak/extract-realm-pubkey.sh`
Onboard a hospital	`./keycloak/onboard-hospital.sh <nhfr-id> "<name>"`
Hospital token	`./keycloak/get-jwt-client-token.sh <nhfr-id>`
Postman token helper	`node keycloak/token-server.mjs` (`:4099`)
JeMPI tunnel up	`sudo wg-quick up wg0`
Push data	`cd FHIR_mediator && python3 main.py` (`--FHIR` for bundles)

Troubleshooting

Symptom	Fix
Console calls fail, network error	accept the cert — https://localhost:8080/heartbeat (2a)
Mediator missing under **Mediators**	`OPENHIM_PASSWORD` ≠ root password (2c)
`.env` change ignored	used `restart`, not `up -d --force-recreate`
`JWT could not be verified`	`realm-public.pem` stale — re-run §5
401 at `:5000` fresh token	issuer mismatch — `iss` must be `http://localhost:8083/realms/openhie`
403 at `:5000`	Client lacks role — re-run `onboard-hospital.sh` (§6)
404 at `:5000/ingest`	`NP FHIR Intake` channel missing — check `fhir-intake-mediator`
403 at `:5000/ingest`	set `OPENHIM_CHANNEL_ROLE=np-fhir-clients` on `fhir-intake-mediator`
422 at `:5000/ingest`	no Organization with NHFR identifier `<nhfr-id>` — `create-org.sh` (§6)
`onboard-hospital.sh` fails at 4/5	Keycloak won't override `sub` — `FACILITY_REGISTRY_JSON` fallback (§6)
container can't reach `10.8.0.1`, host can	masquerade the docker subnet out `wg0` (§3)
Keycloak realm gone after a stop	`docker compose down` was used — use `stop` / `start`